Stratified Event Queue in Verilog

RTL Simulation and Race Conditions

October 12, 2026

Contents

1 Introduction **2**

2 Simulation Time and Event Scheduling **2**

3 Important Event Regions **2**

4 Simplified Event Queue Flow **3**

5 Blocking Assignment and Active Region **3**
5.1 Example 3

6 Nonblocking Assignment and NBA Region **4**
6.1 Example 4

7 Race Condition **4**
7.1 Improved Approach 5

8 RTL Coding Guidelines **5**

9 Summary **6**

1 Introduction

The stratified event queue defines how a Verilog simulator schedules and processes events at a given simulation time.

Multiple statements and processes may execute at the same simulation time, but their scheduling regions determine when values are evaluated and updated.

Understanding this mechanism helps RTL design and verification engineers identify race conditions and write predictable testbenches.

2 Simulation Time and Event Scheduling

Verilog simulation advances through discrete simulation time steps.

At a particular time step, the simulator processes events until the current time slot reaches a stable state. If an event schedules additional work at the same time, the simulator processes that work before advancing to a later time.

For example:

```
initial begin
    a = 1'b0;
    #10 a = 1'b1;
end
```

The value of `a` changes to 1 after 10 simulation time units.

The delay advances simulation time. By contrast, zero-delay assignments and event-triggered processes can cause additional activity within the same time slot.

3 Important Event Regions

The following regions are particularly important for understanding Verilog and SystemVerilog simulation.

Region	**Purpose**
Active	Executes ordinary procedural statements and evaluates right-hand sides of nonblocking assignments.
Inactive	Processes events scheduled by procedural zero-delay controls such as `#0`.
NBA	Applies scheduled nonblocking assignment updates.
Observed	Evaluates concurrent assertions in SystemVerilog.
Reactive	Executes applicable SystemVerilog program-block code and assertion-related reactive behavior.
Postponed	Supports end-of-time-slot observation, including `$strobe` and `$monitor` reporting.

The complete SystemVerilog scheduler contains additional regions and iterative processing rules. The table shows the main regions commonly introduced when teaching RTL simulation.

4 Simplified Event Queue Flow

A simplified view of common simulation activity is:

Active Region
Procedural execution and expression evaluation
↓
Inactive Region
Zero-delay scheduled activity
↓
NBA Region
Nonblocking assignment updates
↓
Re-evaluate affected processes
↓
Observed / Reactive / Postponed
SystemVerilog checks and end-of-slot observation

This is a teaching overview, not a complete scheduler algorithm. The simulator may return to earlier event-processing regions when updates trigger new activity in the same time slot.

5 Blocking Assignment and Active Region

A blocking assignment (=) updates its left-hand side immediately when the statement executes.

5.1 Example

```
module blocking_example;

    reg a, b;

    initial begin
        a = 1'b0;
        b = 1'b0;

        a = 1'b1;
        b = a;

        $display("a=%b b=%b", a, b);
    end

endmodule
```

Output:

```
a=1 b=1
```

Since the blocking assignment updates `a` immediately, the next statement reads the updated value of `a`.

6 Nonblocking Assignment and NBA Region

A nonblocking assignment (<=) evaluates its right-hand side when the statement executes and schedules its left-hand-side update for the NBA region.

6.1 Example

```
module nonblocking_example;

    reg a, b;

    initial begin
        a = 1'b0;
        b = 1'b0;

        a <= 1'b1;
        b <= a;

        #1 $display("a=%b b=%b", a, b);
    end

endmodule
```

Output:

```
a=1 b=0
```

The right-hand side of `b <= a` is evaluated while `a` is still 0. Both updates are then scheduled for the NBA region.

Therefore, `b` receives the old value of `a`.

7 Race Condition

A race condition occurs when the result depends on the order in which concurrent processes execute.

Consider the following example:

```
module race_example;
    reg clk;
    reg q;
    initial clk = 1'b0;
    always #5 clk = ~clk;
    always @(posedge clk)
        q = 1'b1;
    initial begin
        @(posedge clk);
        $display("q=%b", q);
    end
endmodule
```

At the rising clock edge, both the clocked process and the testbench process become eligible to execute in the Active region.

The testbench may read `q` before or after the other process updates it. Consequently, the displayed value is not reliably determined by the intended ordering.

7.1 Improved Approach

For a simple testbench, sample after the active and NBA updates have settled:

```
initial begin
    @(posedge clk);
    #1;
    $display("q=%b", q);
end
```

This example uses a small delay to separate sampling from the clock edge. In a SystemVerilog testbench, clocking blocks or appropriate assertion sampling mechanisms are generally preferable for robust synchronization.

8 RTL Coding Guidelines

- Use nonblocking assignments for clocked sequential logic.
- Use blocking assignments for combinational procedural logic.
- Avoid multiple processes writing the same variable without a well-defined design strategy.
- Avoid relying on execution order between concurrent Active-region processes.
- Synchronize testbench stimulus and sampling with clocking blocks or suitable event controls.
- Use `$display` for immediate reporting and `$strobe` when end-of-time-slot values are required.
- Remember that simulation time does not advance merely because the simulator processes another event region.

9 Summary

- The stratified event queue defines the scheduling and ordering of simulation events.
- Blocking assignments update immediately within procedural execution.
- Nonblocking assignments schedule updates in the NBA region.
- Race conditions arise when concurrent processes have ambiguous ordering.
- Correct assignment styles and testbench synchronization improve simulation reliability.

Key takeaway: Understand event scheduling to predict simulation behavior and avoid race conditions in RTL and testbenches.